

FULL STACK APPLICATION DEVELOPMENT

#SKILL - 9 Global Exception Handling using @ControllerAdvice

Prerequisites:

- Understanding of REST API exception flow
- Concept of custom exceptions

A university application provides student details through REST APIs. When users enter an invalid student ID, the system shows technical error messages that are difficult for users to understand. To improve usability, the application should return clear, readable, and user-friendly error responses through centralized exception handling.

Your task is to implement Global Exception Handling using @ControllerAdvice and create custom exceptions for invalid student inputs.

Tasks:

1. Create a custom exception class named StudentNotFoundException
2. Create a REST endpoint /student/{id} that retrieves student information and throws the custom exception when the given student ID is invalid.
3. Create a GlobalExceptionHandler class using @ControllerAdvice to handle exceptions across the entire application.
4. Add a method with @ExceptionHandler(StudentNotFoundException.class) to return a meaningful error response along with an appropriate HTTP status code.
5. Add a second custom exception, such as InvalidInputException, and handle it inside the same @ControllerAdvice class.
6. Return a structured JSON response (fields like timestamp, message, statusCode) from the exception handler to make the error readable and consistent.
7. Test the API with valid and invalid student IDs and observe the difference in responses.
8. Test an endpoint with invalid input format (e.g., sending text instead of a number) to trigger the second custom exception using Postman.
9. Push the Spring Boot project to GitHub.

Step 1 — Create Spring Boot Project

Open **Spring Tool Suite**, **Eclipse IDE**, or **Visual Studio Code**.

1. Click

File → New → Spring Starter Project

2. Enter project details

Field	Value
Name	studentexception
Type	Maven
Packaging	Jar
Java	17
Group	com
Artifact	klu

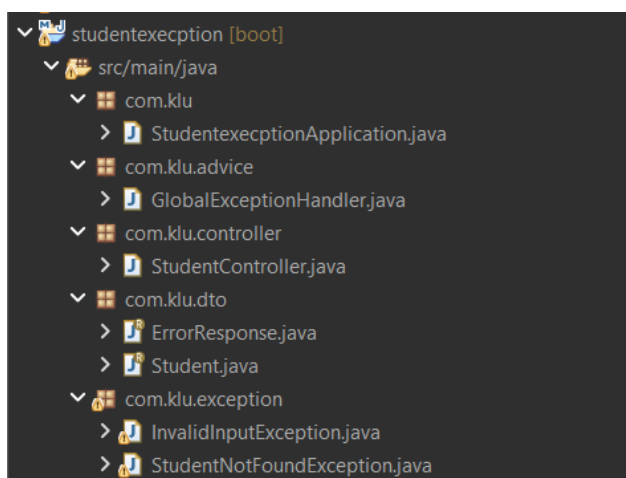
3. Select Dependency

✓ **Spring Web**

Click **Finish**.

The project will be created.

Step 2 — Project Structure



Step 3 — Create Student Class

Name:

Student

Code:

```
package com.klu.dto;

public class Student {

    private int id;

    private String name;

    private String department;

    public Student(int id,String name,String department){

        this.id=id;

        this.name=name;

        this.department=department;

    }

    public int getId(){

        return id;

    }

    public String getName(){

        return name;

    }

    public String getDepartment(){

        return department;

    }

}
```

Step 4 — Create ErrorResponse Class

Class name:

ErrorResponse

Code:

```
package com.klu.dto;
```

```
import java.time.LocalDateTime;
```

```
public class ErrorResponse {
```

```
    private LocalDateTime timestamp;
```

```
    private String message;
```

```
    private int statusCode;
```

```
    public ErrorResponse(LocalDateTime timestamp,String message,int statusCode){
```

```
        this.timestamp=timestamp;
```

```
        this.message=message;
```

```
        this.statusCode=statusCode;
```

```
    }
```

```
    public LocalDateTime getTimestamp(){
```

```
        return timestamp;
```

```
    }
```

```
    public String getMessage(){
```

```
        return message;
```

```
    }
```

```
    public int getStatusCode(){
```

```
        return statusCode;
```

```
    }
```

```
}
```

Step 5 — Create Custom Exception

Class

StudentNotFoundException

Code:

```
package com.klu.exception;
```

```
public class StudentNotFoundException extends RuntimeException{

    public StudentNotFoundException(String message){
        super(message);
    }
}
```

Step 6 — Create Second Custom Exception

Class:

InvalidInputException

Code:

```
package com.klu.exception;
```

```
public class InvalidInputException extends RuntimeException{

    public InvalidInputException(String message){
        super(message);
    }
}
```

Step 7 — Create REST Controller

Class:

StudentController

Code:

```
package com.klu.controller;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import com.klu.dto.Student;
```

```
import com.klu.exception.InvalidInputException;
```

```
import com.klu.exception.StudentNotFoundException;
```

```
@RestController
```

```
@RequestMapping("/student")
```

```
public class StudentController {
```

```
    @GetMapping("/{id}")
```

```
    public Student getStudent(@PathVariable String id){
```

```
        int studentId;
```

```
        try{
```

```
            studentId=Integer.parseInt(id);
```

```
        }
```

```
        catch(NumberFormatException e){
```

```
            throw new InvalidInputException("Student ID must be a number");
```

```
        }
```

```

        if(studentId<=0){
            throw new InvalidInputException("Student ID must be positive");
        }

        if(studentId!=101){
            throw new StudentNotFoundException("Student with ID "+studentId+" not found");
        }

        return new Student(101,"Rishi","CSE");
    }
}

```

Step 8 — Create Global Exception Handler

Class:

GlobalExceptionHandler

Code:

```
package com.klu.advice;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import org.springframework.http.*;
```

```
import com.klu.dto.ErrorResponse;
```

```
import com.klu.exception.InvalidInputException;
```

```
import com.klu.exception.StudentNotFoundException;
```

```
import java.time.LocalDateTime;
```

```
@ControllerAdvice
```

```
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<ErrorResponse>
    handleStudentNotFound(StudentNotFoundException ex){

        ErrorResponse error=new ErrorResponse(
            LocalDateTime.now(),
            ex.getMessage(),
            HttpStatus.NOT_FOUND.value()
        );

        return new ResponseEntity<>(error,HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorResponse> handleInvalidInput(InvalidInputException ex){

        ErrorResponse error=new ErrorResponse(
            LocalDateTime.now(),
            ex.getMessage(),
            HttpStatus.BAD_REQUEST.value()
        );

        return new ResponseEntity<>(error,HttpStatus.BAD_REQUEST);
    }
}
```

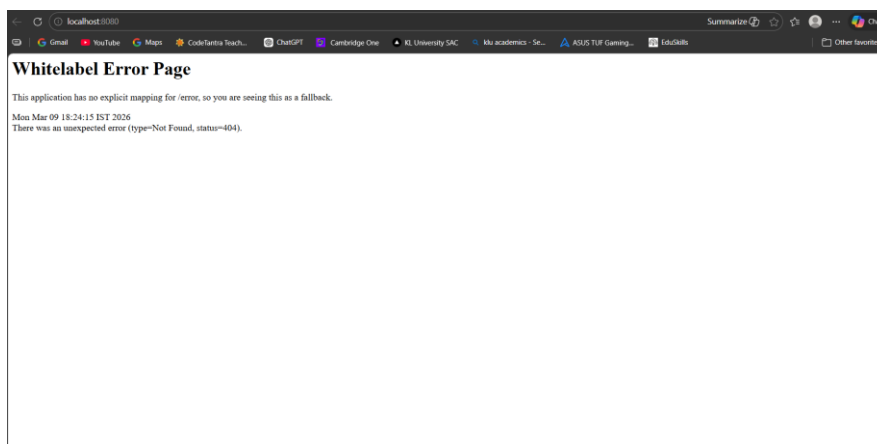
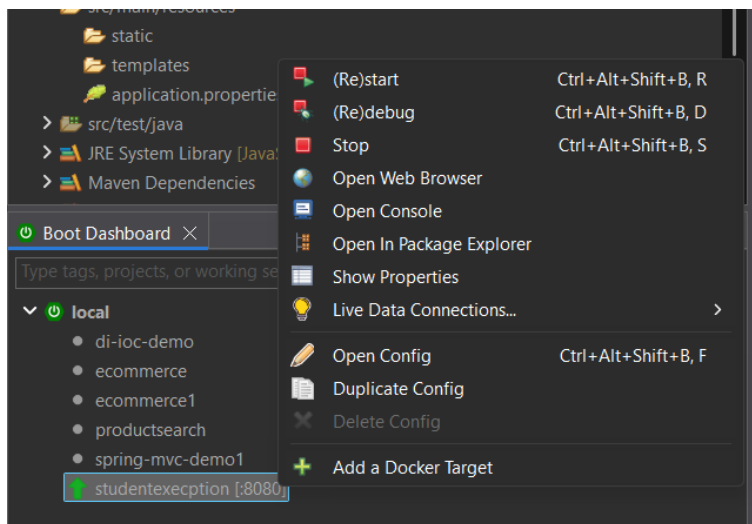

Step 9 — Run Application

Run:

StudentexceptionApplication.java

Server starts:

<http://localhost:8080>



Step 10 — Test Using Postman

Open **Postman**

=>**Test Valid Student**

GET <http://localhost:8080/student/101>

Response:

```
1  {  
2  "id":101,  
3  "name":"Rishi",  
4  "department":"CSE"  
5  }
```

=>**Test Student Not Found**

GET <http://localhost:8080/student/200>

Response:

```
1  {  
2  "timestamp":"2026-03-09T13:20:00",  
3  "message":"Student with ID 200 not found",  
4  "statusCode":404  
5  }
```

=>**Test Invalid Input**

GET <http://localhost:8080/student/abc>

Response:

```
1  {  
2  "timestamp":"2026-03-09T13:21:00",  
3  "message":"Student ID must be a number",  
4  "statusCode":400  
5  }
```